

CL1002-Programming Fundamentals— FALL 2023

LAB 12

Multi-Dimensional Arrays



Learning Outcomes

In this lab you are expected to learn the following:

- 1D Arrays (Revision)
- 2D Arrays

Multi-Dimensional Array:

A multi-dimensional array is an array of arrays.

Declaring a Multi-Dimensional Array:

To declare a multi-dimensional array, define the variable type, specify the name of the array followed by square brackets which specify how many elements the main array has, followed by another set of square brackets which indicates how many elements the sub-arrays have.

For example:

- `int array [3][4];`

	0	1	2	3
0	[0][0]	[0][1]	[0][2]	[0][3]
1	[1][0]	[1][1]	[1][2]	[1][3]
2	[2][0]	[2][1]	[2][2]	[2][3]

- `char Names [2][3]={`

`{ 'A', 'L', 'I' },`

`{ 'Z', 'I', 'A' }`

`};`

Access the Elements of a Multi-Dimensional Array:

To access an element of a multi-dimensional array, specify an index number in each of the array's dimensions. This statement accesses the value of the element in the first row (0) and third column (2) of the letters array.

For example:

- ```
string letters = {
 { "A", "B", "C", "D" },
 { "E", "F", "G", "H" }
};

cout << letters[0][2]; // Outputs "C"
```

### Change Elements in a Multi-Dimensional Array:

To change the value of an element, refer to the index number of the element in each of the dimensions.

For example:

```
• string letters[2][4] = {
 { "A", "B", "C", "D" },
 { "E", "F", "G", "H" }
};
letters[0][0] = "Z";
cout << letters[0][0]; // Now outputs "Z" instead of "A"
```

#### Loop Through a Multi-Dimensional Array:

To loop through a multi-dimensional array, you need one loop for each of the array's dimensions. The following example outputs all elements in the letters array.

For example:

```
• string letters[2][4] = {
 { "A", "B", "C", "D" },
 { "E", "F", "G", "H" }
};

for (int i = 0; i < 2; i++) {
 for (int j = 0; j < 4; j++) {
 cout << letters[i][j] << "\n";
 }
}
```

#### Example Programs Section

##### Example Program#01: Using Two-Dimensional Array

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4 int test[3][2] = {{2, -5}, {4, 0}, {9, 1}};
5 // use of nested for loop
6 // access rows of the array
```



```
7 for (int i = 0; i < 3; ++i) {
8 // access columns of the array
9 for (int j = 0; j < 2; ++j) {
10 cout << "test[" << i << "][" << j << "] = " << test[i][j] << endl; }
11 }
12 return 0;
13 }
```

**Output:**

```
test[0][0] = 2
test[0][1] = -5
test[1][0] = 4
test[1][1] = 0
test[2][0] = 9
test[2][1] = 1
```

Example Program#02: Entering string [SPACES MAY NOT WORK]

```
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5 char str[100];
6 cout << "Enter a string: ";
7 cin >> str;
8 cout << "You entered: " << str << endl;
9 return 0;
10 }
```

**Output:**

```
Enter a string: Muhammad Iqbal
You entered: Muhammad
```



Example Program#03: Entering string [ACCEPTING SPACES]

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4 char str[100];
5 cout << "Enter a string: ";
6 cin.get(str, 100);
7 cout << "You entered: " << str << endl;
8 return 0;
9 }
```

**Output:**

Enter a string: My name is C++

You entered: My name is C++

Example Program#04: Passing array as argument to a function

```
1 #include <iostream>
2 using namespace std;
3 void displayArray(int array[], int size){
4 for(int i=0; i<size; i++){
5 cout<<array[i]<<" ";
6 }
7 cout<<endl;
8 }
9 int main()
10 {
11 int theArray[]={2,4,6,7,8,9};
12 int theSize = sizeof(theArray)/sizeof(theArray[0]);
13 displayArray(theArray, theSize);
14 }
```

**Output:**

2 4 6 7 8 9

**Submission Instructions:**

1. Save the **cpp** file with the roll no and task number  
e.g. i230001\_Q1.cpp
2. Now create a new folder with *ROLLNO\_LAB11\_SEC* e.g. i23XXXX\_LAB11\_A
3. You need to display your roll no and name before the output of each question.
4. Now you have to submit this zipped file on Google Classroom.
5. If you don't follow the above-mentioned submission instructions, you will be marked **zero**.
6. Plagiarism in the Lab Task will result in **zero** marks in the whole category.

## Lab Tasks

### Task 01

Write a C++ program to determine whether a given character array is a palindrome. A palindrome is a sequence of characters that reads the same forwards and backward. You need to declare a character array with a constant size of 20 and take the input from the user.

**Sample Output:**

```
Enter a character array: racecar
The entered character array is a palindrome.

Enter a character array: Hello
The entered character array is not a palindrome.
```

### Task 02

Declare a character array of constant size  $n = 25$  and store the read input in it.

1. Check and print the vowels in the array while traversing it. Use the function with the following prototype to check if the character is a vowel.  
**bool checkVowels (char a);**
2. Now traverse the array and display the count of the character (given by the user) repeated in the array.

**Sample Output:**

```
Character array is: programming fundamentals
Returned array has: oaive
Count(m) is: 3
Count(m) returned: 1
Count(x) returned: 0
```

### Task 03

Take an array of length  $n$  of constant size 5 where all the numbers are nonnegative and unique. Find the element in the array possessing the highest value. Split the element into two parts where the first part contains the next highest value in the array. And, the second part holds the required additive entity to get the highest value. Display the array before and after the split is performed.

```
Before Split: 4 8 6 3 2
After Split: 4 2 6 6 3 2
```

### Task 04

Write a C++ program that searches for a given element in a 2D array and returns its position.

**Sample Output:**

```
Enter the number of rows: 3
Enter the number of columns: 4
Enter the matrix elements:
2 5 8 9
4 7 11 13
6 10 15 18
Enter the value to search for: 11
The value 11 is found at position (1, 2).
```

### Task 05

Write a C++ program that takes two matrices as input and performs matrices multiplication and displays the resultant matrix.

#### Compatibility of Dimensions:

The number of columns in the first matrix must be equal to the number of rows in the second matrix. In other words, if the first matrix is of size  $m \times n$ , the second matrix must be of size  $n \times p$ . This ensures that the inner dimensions match.

#### Resulting Matrix Size:

The resulting matrix will have dimensions of  $m \times p$ , where " $m$ " is the number of rows in the first matrix, and " $p$ " is the number of columns in the second matrix.

**Sample Output:**

```
Enter the number of rows (m) and columns (n) for the first matrix:
m1 = 2
n1 = 3
Enter the number of rows (m) and columns (n) for the second matrix:
m2 = 3
n2 = 2
```

```
Enter the elements of the first matrix (2x3):
Enter element at position (1,1): 2
Enter element at position (1,2): 3
Enter element at position (1,3): 4
Enter element at position (2,1): 5
Enter element at position (2,2): 6
Enter element at position (2,3): 7
Enter the elements of the second matrix (3x2):
Enter element at position (1,1): 1
Enter element at position (1,2): 2
Enter element at position (2,1): 3
Enter element at position (2,2): 4
Enter element at position (3,1): 5
Enter element at position (3,2): 6
Matrix Multiplication (Result):
26 34
59 76
```

Happy Coding!